LABORATORY WORK 5

Part 1: CHECK Constraints

Task 1.1

```
CREATE TABLE employees (
    employee_id INTEGER,
    first_name TEXT,
    last_name TEXT,
    age INTEGER CHECK (age BETWEEN 18 AND 65),
    salary NUMERIC CHECK (salary > 0)
);

INSERT INTO employees VALUES
(1, 'John', 'Smith', 25, 5000),
(2, 'Sarah', 'Johnson', 30, 6000);

-- Invalid: age must be between 18 and 65
-- INSERT INTO employees VALUES (3, 'Tom', 'Brown', 16, 4000);

-- Invalid: salary must be greater than 0
-- INSERT INTO employees VALUES (4, 'Anna', 'White', 25, -100);
```

Task 1.2

```
CREATE TABLE products_catalog (
    product_id INTEGER,
    product_name TEXT,
    regular_price NUMERIC,
    discount_price NUMERIC,
    CONSTRAINT valid_discount CHECK (
        regular_price > 0
        AND discount_price > 0
        AND discount_price < regular_price
    )
);

INSERT INTO products_catalog VALUES
(1, 'Laptop', 1500, 1200),
(2, 'Phone', 800, 650);

-- Invalid: discount_price must be less than regular_price
-- INSERT INTO products_catalog VALUES (3, 'Tablet', 500, 600);

-- Invalid: regular_price must be greater than 0
-- INSERT INTO products_catalog VALUES (4, 'Watch', 0, 100);

-- Invalid: discount_price must be greater than 0
-- INSERT INTO products_catalog VALUES (5, 'Mouse', 100, 0);
```

Task 1.3

```
CREATE TABLE bookings (
    booking_id INTEGER,
    check_in_date DATE,
    check_out_date DATE,
    num_guests INTEGER CHECK (num_guests BETWEEN 1 AND 10),
    CHECK (check_out_date > check_in_date)
);

INSERT INTO bookings VALUES
(1, '2026-10-15', '2026-10-18', 2),
(2, '2026-11-01', '2026-11-05', 4);
```

```
-- Invalid: num_guests must be between 1 and 10
-- INSERT INTO bookings VALUES (3, '2026-10-15', '2026-10-18', 0);

-- Invalid: check_out_date must be after check_in_date
-- INSERT INTO bookings VALUES (4, '2026-10-18', '2026-10-15', 2);
```

Task 1.4: Testing CHECK Constraints

The valid INSERT statements above successfully insert two rows into each table. The commented-out INSERT statements demonstrate violations of the CHECK constraints. PostgreSQL rejects these invalid records.

Part 2: NOT NULL Constraints

Task 2.1

```
CREATE TABLE customers (
    customer_id INTEGER NOT NULL,
    email TEXT NOT NULL,
    phone TEXT,
    registration_date DATE NOT NULL
);

INSERT INTO customers VALUES
(1, 'john@example.com', '123456789', '2026-01-10'),
(2, 'sarah@example.com', NULL, '2026-02-15');

-- Invalid: email cannot be NULL
-- INSERT INTO customers VALUES (3, NULL, '987654321', '2026-03-01');

-- Invalid: registration_date cannot be NULL
-- INSERT INTO customers VALUES (4, 'tom@example.com', '111111111', NULL);

-- Valid: phone can be NULL
INSERT INTO customers VALUES
(5, 'lisa@example.com', NULL, '2026-03-10');
```

Task 2.2

```
CREATE TABLE inventory (
    item_id INTEGER NOT NULL,
    item_name TEXT NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity >= 0),
    unit_price NUMERIC NOT NULL CHECK (unit_price > 0),
    last_updated TIMESTAMP NOT NULL
);

INSERT INTO inventory VALUES
(1, 'Laptop', 10, 1200, CURRENT_TIMESTAMP),
(2, 'Mouse', 50, 25, CURRENT_TIMESTAMP);

-- Invalid: item_name cannot be NULL
-- INSERT INTO inventory VALUES (3, NULL, 10, 50, CURRENT_TIMESTAMP);

-- Invalid: quantity cannot be NULL
-- INSERT INTO inventory VALUES (4, 'Keyboard', NULL, 40, CURRENT_TIMESTAMP);

-- Invalid: unit_price cannot be NULL
-- INSERT INTO inventory VALUES (5, 'Monitor', 5, NULL, CURRENT_TIMESTAMP);

-- Invalid: last_updated cannot be NULL
-- INSERT INTO inventory VALUES (6, 'USB Cable', 20, 10, NULL);
```

```
-- Invalid: quantity cannot be negative
-- INSERT INTO inventory VALUES (7, 'Tablet', -1, 100, CURRENT_TIMESTAMP);

-- Invalid: unit_price must be greater than 0
-- INSERT INTO inventory VALUES (8, 'Camera', 5, 0, CURRENT_TIMESTAMP);
```

Task 2.3: Testing NOT NULL

The valid INSERT statements demonstrate complete records and NULL values in the nullable phone column. The commented-out statements demonstrate NOT NULL and CHECK constraint violations.

Part 3: UNIQUE Constraints

Task 3.1

```
CREATE TABLE users (
    user_id INTEGER,
    username TEXT UNIQUE,
    email TEXT UNIQUE,
    created_at TIMESTAMP
);

INSERT INTO users VALUES
(1, 'john123', 'john@example.com', CURRENT_TIMESTAMP),
(2, 'sarah123', 'sarah@example.com', CURRENT_TIMESTAMP);
```

Task 3.2

```
CREATE TABLE course_enrollments (
    enrollment_id INTEGER,
    student_id INTEGER,
    course_code TEXT,
    semester TEXT,
    CONSTRAINT unique_enrollment
        UNIQUE (student_id, course_code, semester)
);

INSERT INTO course_enrollments VALUES
(1, 101, 'CS101', 'Fall 2026'),
(2, 101, 'CS102', 'Fall 2026'),
(3, 102, 'CS101', 'Fall 2026');

-- Invalid: duplicate student, course, and semester combination
-- INSERT INTO course_enrollments VALUES
-- (4, 101, 'CS101', 'Fall 2026');
```

Task 3.3: Named UNIQUE Constraints

```
ALTER TABLE users
ADD CONSTRAINT unique_username UNIQUE (username);

ALTER TABLE users
ADD CONSTRAINT unique_email UNIQUE (email);
```

The table created in Task 3.1 already has unnamed UNIQUE constraints on both columns. To avoid duplicate constraints, use the following alternative when creating the table initially:

```
CREATE TABLE users_named (
    user_id INTEGER,
    username TEXT,
    email TEXT,
    created_at TIMESTAMP,
    CONSTRAINT unique_username UNIQUE (username),
    CONSTRAINT unique_email UNIQUE (email)
```

```
);

INSERT INTO users_named VALUES
(1, 'john123', 'john@example.com', CURRENT_TIMESTAMP),
(2, 'sarah123', 'sarah@example.com', CURRENT_TIMESTAMP);

-- Invalid: duplicate username
-- INSERT INTO users_named VALUES
-- (3, 'john123', 'tom@example.com', CURRENT_TIMESTAMP);

-- Invalid: duplicate email
-- INSERT INTO users_named VALUES
-- (4, 'tom123', 'john@example.com', CURRENT_TIMESTAMP);
```

Part 4: PRIMARY KEY Constraints

Task 4.1

```
CREATE TABLE departments (
    dept_id INTEGER PRIMARY KEY,
    dept_name TEXT NOT NULL,
    location TEXT
);

INSERT INTO departments VALUES
(1, 'IT', 'Almaty'),
(2, 'HR', 'Astana'),
(3, 'Sales', 'Shymkent');

-- Invalid: duplicate primary key
-- INSERT INTO departments VALUES (1, 'Finance', 'Almaty');

-- Invalid: primary key cannot be NULL
-- INSERT INTO departments VALUES (NULL, 'Marketing', 'Astana');
```

Task 4.2

```
CREATE TABLE student_courses (
    student_id INTEGER,
    course_id INTEGER,
    enrollment_date DATE,
    grade TEXT,
    PRIMARY KEY (student_id, course_id)
);

INSERT INTO student_courses VALUES
(1, 101, '2026-09-01', 'A'),
(1, 102, '2026-09-02', 'B'),
(2, 101, '2026-09-03', 'A');

-- Invalid: duplicate composite primary key
-- INSERT INTO student_courses VALUES
-- (1, 101, '2026-09-10', 'C');
```

Task 4.3: Comparison Exercise

1. **UNIQUE vs PRIMARY KEY:** A PRIMARY KEY uniquely identifies each row and cannot contain NULL values. A UNIQUE constraint prevents duplicate values but allows NULL values in PostgreSQL.
2. **Single-column vs composite PRIMARY KEY:** A single-column primary key identifies a row using one column. A composite primary key uses two or more columns when their combination uniquely identifies a row.

3. **One PRIMARY KEY but multiple UNIQUE constraints:** A table has only one primary key definition because it has one designated primary identifier. It can have multiple UNIQUE constraints to enforce uniqueness on other columns or combinations of columns.

Part 5: FOREIGN KEY Constraints

Task 5.1

```
CREATE TABLE employees_dept (
    emp_id INTEGER PRIMARY KEY,
    emp_name TEXT NOT NULL,
    dept_id INTEGER REFERENCES departments(dept_id),
    hire_date DATE
);

INSERT INTO employees_dept VALUES
(1, 'John Smith', 1, '2024-01-15'),
(2, 'Sarah Johnson', 2, '2024-03-20'),
(3, 'Michael Brown', 3, '2025-02-10');

-- Invalid: department 99 does not exist
-- INSERT INTO employees_dept VALUES
-- (4, 'Tom White', 99, '2025-05-10');
```

Task 5.2: Library Database

```
CREATE TABLE authors (
    author_id INTEGER PRIMARY KEY,
    author_name TEXT NOT NULL,
    country TEXT
);

CREATE TABLE publishers (
    publisher_id INTEGER PRIMARY KEY,
    publisher_name TEXT NOT NULL,
    city TEXT
);

CREATE TABLE books (
    book_id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    author_id INTEGER REFERENCES authors(author_id),
    publisher_id INTEGER REFERENCES publishers(publisher_id),
    publication_year INTEGER,
    isbn TEXT UNIQUE
);

INSERT INTO authors VALUES
(1, 'George Orwell', 'United Kingdom'),
(2, 'J.K. Rowling', 'United Kingdom'),
(3, 'Paulo Coelho', 'Brazil'),
(4, 'Jane Austen', 'United Kingdom'),
(5, 'Abai Kunanbayev', 'Kazakhstan');

INSERT INTO publishers VALUES
(1, 'Penguin Books', 'London'),
(2, 'Bloomsbury', 'London'),
(3, 'HarperCollins', 'New York'),
(4, 'Vintage', 'New York'),
(5, 'Kazakh Press', 'Almaty');

INSERT INTO books VALUES
(1, '1984', 1, 1, 1949, '9780000000001'),
```

```
(2, 'Harry Potter', 2, 2, 1997, '9780000000002'),
(3, 'The Alchemist', 3, 3, 1988, '9780000000003'),
(4, 'Pride and Prejudice', 4, 4, 1813, '9780000000004'),
(5, 'Words of Edification', 5, 5, 1890, '9780000000005');
```

Task 5.3: ON DELETE Options

```
CREATE TABLE categories (
    category_id INTEGER PRIMARY KEY,
    category_name TEXT NOT NULL
);

CREATE TABLE products_fk (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    category_id INTEGER REFERENCES categories(category_id)
        ON DELETE RESTRICT
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    order_date DATE NOT NULL
);

CREATE TABLE order_items (
    item_id INTEGER PRIMARY KEY,
    order_id INTEGER REFERENCES orders(order_id)
        ON DELETE CASCADE,
    product_id INTEGER REFERENCES products_fk(product_id),
    quantity INTEGER CHECK (quantity > 0)
);

INSERT INTO categories VALUES
(1, 'Electronics'),
(2, 'Books');

INSERT INTO products_fk VALUES
(1, 'Laptop', 1),
(2, 'Keyboard', 1),
(3, 'Notebook', 2);

INSERT INTO orders VALUES
(1, '2026-10-01'),
(2, '2026-10-05');

INSERT INTO order_items VALUES
(1, 1, 1, 2),
(2, 1, 2, 1),
(3, 2, 3, 5);

-- Test 1: This DELETE fails because category 1 has products.
-- DELETE FROM categories WHERE category_id = 1;

-- Test 2: This DELETE automatically removes order_items for order 1.
DELETE FROM orders WHERE order_id = 1;

-- Test results:
-- RESTRICT prevents deleting a category referenced by products.
-- CASCADE automatically deletes order_items referencing a deleted order.
```

Part 6: Practical Application

Task 6.1: E-commerce Database Design

```
-- Create customers table.
```

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    registration_date DATE NOT NULL
);

-- Create products table.
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    description TEXT,
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    stock_quantity INTEGER NOT NULL CHECK (stock_quantity >= 0)
);

-- Create orders table.
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL
        REFERENCES customers(customer_id) ON DELETE RESTRICT,
    order_date DATE NOT NULL,
    total_amount NUMERIC(12,2) NOT NULL CHECK (total_amount >= 0),
    status TEXT NOT NULL CHECK (
        status IN (
            'pending', 'processing', 'shipped',
            'delivered', 'cancelled'
        )
    )
);

-- Create order_details table.
CREATE TABLE order_details (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL
        REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id INTEGER NOT NULL
        REFERENCES products(product_id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0)
);

-- Insert 5 customers.
INSERT INTO customers VALUES
(1, 'John Smith', 'john@example.com', '111111111', '2026-01-10'),
(2, 'Sarah Johnson', 'sarah@example.com', '222222222', '2026-02-15'),
(3, 'Michael Brown', 'michael@example.com', '333333333', '2026-03-01'),
(4, 'Emily Davis', 'emily@example.com', '444444444', '2026-04-05'),
(5, 'Robert Wilson', 'robert@example.com', '555555555', '2026-05-20');

-- Insert 5 products.
INSERT INTO products VALUES
(1, 'Laptop', 'Business laptop', 1200.00, 10),
(2, 'Mouse', 'Wireless mouse', 25.00, 50),
(3, 'Keyboard', 'Mechanical keyboard', 75.00, 30),
(4, 'Monitor', '27-inch monitor', 300.00, 15),
(5, 'Headphones', 'Wireless headphones', 100.00, 20);

-- Insert 5 orders.
INSERT INTO orders VALUES
(1, 1, '2026-09-01', 1225.00, 'pending'),
(2, 2, '2026-09-02', 75.00, 'processing'),
(3, 3, '2026-09-03', 300.00, 'shipped'),
```

```
(4, 4, '2026-09-04', 100.00, 'delivered'),
(5, 5, '2026-09-05', 150.00, 'cancelled');

-- Insert 5 order details.
INSERT INTO order_details VALUES
(1, 1, 1, 1, 1200.00),
(2, 1, 2, 1, 25.00),
(3, 2, 3, 1, 75.00),
(4, 3, 4, 1, 300.00),
(5, 4, 5, 1, 100.00);

-- Test 1: Duplicate email violates UNIQUE.
-- INSERT INTO customers VALUES
-- (6, 'Test User', 'john@example.com', '666666666', '2026-06-01');

-- Test 2: Negative price violates CHECK.
-- INSERT INTO products VALUES
-- (6, 'Bad Product', 'Invalid price', -10.00, 5);

-- Test 3: Negative stock violates CHECK.
-- INSERT INTO products VALUES
-- (7, 'Bad Stock', 'Invalid quantity', 10.00, -1);

-- Test 4: Invalid status violates CHECK.
-- INSERT INTO orders VALUES
-- (6, 1, '2026-09-06', 100.00, 'unknown');

-- Test 5: Non-existent customer violates FOREIGN KEY.
-- INSERT INTO orders VALUES
-- (7, 99, '2026-09-07', 50.00, 'pending');

-- Test 6: Zero quantity violates CHECK.
-- INSERT INTO order_details VALUES
-- (6, 1, 2, 0, 25.00);

-- Test 7: Non-existent product violates FOREIGN KEY.
-- INSERT INTO order_details VALUES
-- (7, 1, 99, 1, 50.00);

-- Test 8: Negative unit price violates CHECK.
-- INSERT INTO order_details VALUES
-- (8, 1, 2, 1, -25.00);

-- Test results:
-- PRIMARY KEY prevents duplicate or NULL identifiers.
-- FOREIGN KEY ensures referenced records exist.
-- CHECK prevents invalid prices, stock quantities, statuses, and order
quantities.
-- UNIQUE prevents duplicate customer email addresses.
-- NOT NULL ensures required fields contain values.
-- ON DELETE CASCADE deletes order details when an order is deleted.
-- ON DELETE RESTRICT prevents deleting customers or products referenced by
orders.
```

Name: Naskhat Maman

Student ID: 25B031364